PRA-vLLM: Separating Query-Addressed Logical Memory from Paged KV Placement

Jorge Simão
EInnovator R&D

August 2026

Abstract

vLLM virtualizes where sequential key–value (KV) blocks reside. Progressive Retrieval Attention (PRA) virtualizes a different choice: which subset of a logically large, non-prefix memory is active for a query. We define a capability-honest integration boundary between these mappings, implement a stable logical PRA block namespace and residency state machine, and implement a vLLM-Metal store that places selected K/V in native paged-cache blocks. On an Apple M5 with vLLM 0.28.0 and the vLLM-Metal plugin, a 25-request smoke study separates exact-prefix reuse from selected-memory recovery. A selected 365-token prompt and the 1,577-token full context both recover the codeword in 5/5 requests; the selected condition uses 76.9% fewer visible tokens and reduces cold time to first token (TTFT) from 776.6 to 80.7 ms. Warm TTFT is 46.2 versus 52.9 ms, while vLLM reports an aggregate prefix-cache hit rate of 75.3–83.4%. These measurements establish a strong black-box baseline. Separately, 80 controlled kernel measurements over five seeds, 16–512 selected tokens, and concurrency 1–8 match dense attention within maximum absolute error 0.00391. Warm native paged attention rises from 0.489 to 0.642 ms; immutable block sharing avoids up to 14.0 MB at concurrency eight. A subsequent five-seed live V1 experiment reserves scheduler-invisible tail pages and prepends them only to attention block tables. Native and full context recover 5/5 answers, disabled memory recovers 0/5, cleanup leaks are 0/5, and wrong memory controls follow the wrong fact in 5/5 cases. This closes controlled V1 generation on Metal, but not APC coexistence in the same native request, serving concurrency, connectors, or CUDA integration.

1 Introduction

Long-running LLM services face two related but distinct memory problems. Sequential request KV must be allocated, paged, reused, and evicted. Separately, a retained corpus or session history need not participate in every query. The first problem is physical. The second is logical.

PagedAttention and vLLM address the physical problem by decoupling logical sequence positions from non-contiguous cache blocks and scheduling requests over the resulting pool [1]. PRA addresses the logical problem by separating address state from detail state and activating only selected non-prefix memory. The combined mapping is

$$\text{logical PRA memory} \rightarrow \text{query-selected logical blocks} \rightarrow \text{physical engine blocks}.$$

The middle arrow cannot be inferred from ordinary prefix equality. Two requests may share the same resource but select different intervals; two physical cache slots may hold the same immutable detail while using request-local position metadata.

This paper asks whether exposing that middle mapping to vLLM improves the quality-adjusted memory, latency, and throughput frontier beyond an optimized route-then-materialize baseline. The current iteration makes three contributions:

1. a stable block identity and validated residency lifecycle independent of vLLM's physical block numbers;
2. a vLLM adapter whose advertised PRA level is derived from an installed native executor, with tenant-scoped prefix-cache salt support at E0;
3. native Metal paged-K/V kernel and live V1 generation studies, alongside the selected-text serving baseline, with parity, causality, cleanup, latency, and physical-sharing controls.

2 Two Independent Reuse Systems

Automatic Prefix Caching (APC) reuses an exact token prefix. PRA selects a non-prefix logical resource or interval. These mechanisms can coexist:

$$\underbrace{\text{stable conversation prefix}}_{\text{APC}} + \underbrace{\text{query-selected retained memory}}_{\text{PRA}}.$$

The security scopes are also independent. Prefix cache presence does not grant resource authorization, and a PRA block selected for one tenant cannot be reused by another merely because its physical bytes remain resident.

Our E0/G10 baseline performs selection before the engine, injects the selected text into the current turn, and lets ordinary vLLM execute the request. This is the baseline a deeper integration must beat. Paged KV being active inside vLLM does not by itself make this path native PRA.

3 Logical PRA Block Contract

The implementation in `src/pra_hf/engine_memory.py` defines identity with the following fields:

Field group	Meaning
scope	tenant, optional session, and authorization scope
resource	resource identifier, immutable version, and record type
interval	half-open source-token interval $[i, j)$
model	immutable model revision, consumer layer, dtype, and K/V layout
policy	materialization profile and source-position/rebinding policy

The canonical JSON representation is hashed to produce an engine-safe logical key. Physical vLLM block numbers are deliberately absent. A resource update invalidates every matching physical realization without changing the identity of unrelated versions.

The state machine is

$$\textsc{IndexedOnly} \to \textsc{OffDevice} \to \textsc{Prefetching} \to \textsc{Resident},$$
$$\textsc{Resident} \leftrightarrow \textsc{Pinned}, \qquad \textsc{Resident} \to \textsc{Evictable} \to \textsc{OffDevice},$$

with an explicit terminal Invalid state. A transition to resident or pinned is rejected unless the engine bridge supplies at least one physical handle. Selection checks tenant and authorization scope before incrementing reuse counters. Snapshots report address bytes, logical detail bytes, resident detail bytes, indexed-only detail bytes, selections, and reuses separately.

4 Capability-Honest vLLM Adapter

The adapter in `src/pra_vllm/adapter.py` has two forms. Without a native executor it is an E0 OpenAI-compatible facade. It advertises APC and text fallback, but *not* logical references or native K/V. A deployment secret and tenant identifier derive a SHA-256 `cache_salt`; neither value is placed directly in the request.

With a native executor, the adapter becomes E2 and delegates generation, streaming, session close, and block-store access to that executor. This dependency-injection boundary is intentional. Metadata registration alone cannot promote the capability matrix. An executor must map selected identities to physical blocks and preserve K/V layout, GQA/MQA, masks, positions, consumer layers, request lifetime, and one correct attention normalization.

The concrete bridge is implemented in the vLLM-Metal native module. It maps each immutable logical key to ordinary native paged-cache pages and packs post-position K/V in $[\text{block}, \text{token}, H_{kv}, D]$ layout, builds one block table per request, and invokes the native Metal `paged_attention_primitive`. Multiple requests may reference one physical handle. This is the verified attention-kernel foundation used by the live V1 bridge below.

The next integration layer is implemented in `src/pra_vllm/v1_metadata.py`. A request registers immutable logical keys, selected block tables per cache group, selected-token count, source-position base, and consumer layers. At each V1 step the registry combines that object with scheduler-owned local cache starts and query-token counts without rewriting either. Thus two quantities remain explicit:

$$p_{q,0} = p_{\text{source}} + T_{\text{local}}, \qquad T_{\text{attn}} = T_{\text{selected}} + T_{\text{local}} + T_q.$$

Table 1: vLLM-Metal smoke results. TTFT is milliseconds. "Warm cached" is not available per request in the SSE usage payload.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	875.9	42.2	n/a
Prefix	0%	333	116.6	49.4	n/a
Selected	100%	56	62.7	42.9	n/a
Prefix + selected	100%	365	80.7	46.2	n/a
Full context	100%	1577	776.6	52.9	n/a

Appending selected pages to ordinary scheduler block tables would instead make V1 count them as sequential prefix state and assign the wrong query positions. The registry validates immutable request registration and explicit cleanup. `src/pra_vllm/v1_native.py` supplies the corresponding model-runner hook: it extends the physical Metal cache with a reserved tail, writes immutable selected pages there, and prepends those page IDs only to request-local attention metadata. Scheduler slot mappings, sequential token counts, and ordinary prefix identities remain unchanged. Query positions advance by the source-position base. This first E2 path requires page-aligned selections, one cache group, all attention layers, and non-TurboQuant K/V.

5 Environment and Method

The remote host was an Apple M5 MacBook Pro with 16 GB unified memory, macOS 26.4.1, and Metal 4. The environment used Python 3.12.7, vLLM 0.28.0+cpu, and vLLM-Metal 0.3.0.dev20260828085745 at revision `14705ad974863f68d00315655514f200366441bf`. Although the package version contains `+cpu`, startup logs show the Metal plugin, MLX model loader, and native M5 paged-attention kernels. We therefore report both package and backend provenance.

The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The server used a 2,048-token maximum context, one sequence, 20% memory reservation, and explicit APC. It allocated 1,589.1 MB for 866 KV blocks of 16 tokens, or 13,856 tokens.

The fixed synthetic task contains one authoritative codeword and twelve distractor records. Five conditions separate no memory, a stable prefix, selected evidence, stable prefix plus selected evidence, and full context. Each condition is repeated five times contiguously. The first request is reported as cold within the running server; the remaining four form the warm mean. Streaming Server-Sent Events determine TTFT. Twenty-five requests are enough for compatibility and mechanism smoke evidence, not tail percentiles or serving claims.

6 Results

Table 1 first validates the control. Neither no-memory condition recovers the codeword, whereas selected and full context recover it in every request. Prefix plus selected memory uses 365 prompt tokens versus 1,577 for full context, a reduction of

$$1 - \frac{365}{1577} = 76.9\%.$$

Cold TTFT is 80.7 ms for prefix plus selected memory and 776.6 ms for full context. This large difference includes first-touch effects and should not be treated as a steady-state speedup. Warm TTFT narrows to 46.2 versus 52.9 ms. The engine log reports aggregate APC hit rates of 75.3% and 83.4% during the run, demonstrating that selected context did not disable prefix caching.

Figure 1 shows the stable prompt-size reduction across all three tokenizers and the engine-specific latency scale. The comparable result is within-engine selected versus full context, not the absolute height of different-engine bars.

6.1 Native paged selected K/V

Table 2 tests the physical mechanism directly. Selected K/V occupies native vLLM-Metal pages and is consumed by the same paged-attention primitive used for sequential cache blocks. The output agrees with a dense MLX reference at every tested geometry. Warm latency changes smoothly with selected set size after compilation, and one immutable allocation is shared by all requests. These are positive block-execution and sharing results; the next experiment tests their use during generation.

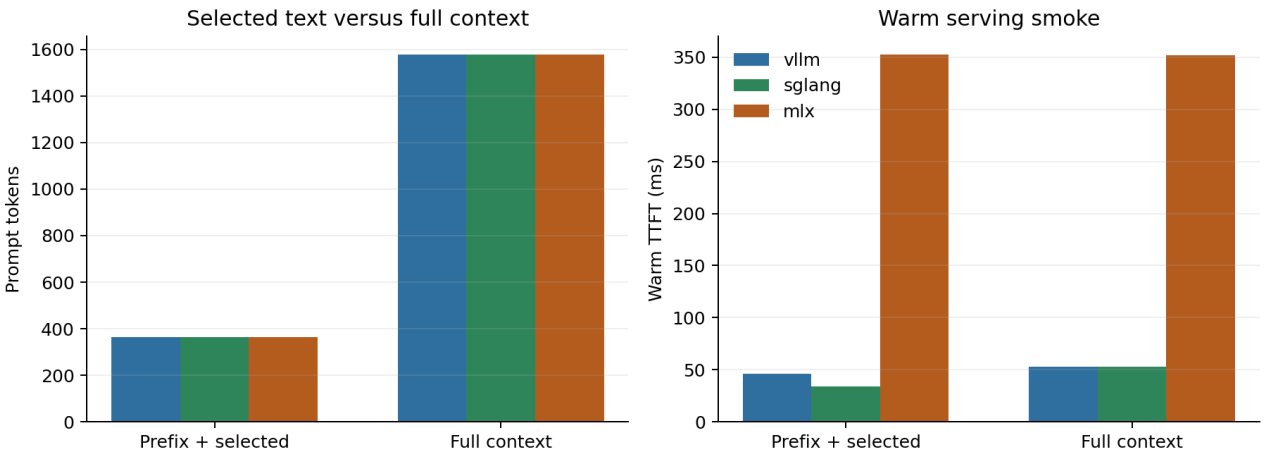

Figure 1: Shared prompt-size and warm-TTFT calibration. Timings across engines are not direct speed rankings: SGLang used an unquantized MLX conversion, whereas vLLM and MLX-LM used the 4-bit checkpoint.

Table 2: Native vLLM-Metal paged attention. Each row aggregates five seeds and four concurrency levels. Cold includes first invocation for that row; warm is an immediate repeat. Sharing savings are at concurrency eight.

Selected tok.	Fraction	Cold ms	Warm ms	Max error	Shared MB saved
16	0.20%	2.769	0.489	0.003906	0.44
64	0.78%	0.602	0.523	0.0009766	1.75
256	3.12%	0.675	0.548	0.0009766	7.00
512	6.25%	0.722	0.642	0.0007324	14.00

6.2 Live V1 generation

Table 3 carries 32 selected native tokens through actual V1 generation while only 15 query tokens are visible. Native selected K/V and full visible context both recover the target in all five seeds. Query-only generation never recovers it, the request after native cleanup shows no leak, and substituting a wrong native memory changes the answer to that memory's code in every seed. Mean completion time is 159.7 ms for native memory and 197.3 ms for full context; this small controlled cohort is a mechanism timing, not a throughput claim. Active selected K/V is 3.50 MiB. The experiment closes generation, causal influence, and request cleanup on this pinned Metal path.

6.3 Controlled residency pressure

The engine-neutral residency manager was also exercised on five deterministic 120-request traces with a 14 MB budget and a 36 MB working set. This is a control-plane simulation; it does not substitute for engine telemetry. Size-aware LRU reduces mean reload amplification from 0.643 to 0.397, a 38.3% relative reduction. Reuse-count and reload-cost policies reach 0.445 and 0.437. The result confirms that selected-memory lifecycle policy can matter even before learned eviction, while leaving its effect on real TTFT open.

Table 3: Five-seed live vLLM-Metal V1 generation controls. "Control success" means recovery for positive conditions, failure for disabled memory, absence of post-request leakage, or following the deliberately wrong memory.

Condition	Control success	Mean latency ms
Full visible context	100%	197.3
Selected native K/V	100%	159.7
Disabled memory (must fail)	100%	–
Post-cleanup isolation	100%	–
Wrong-memory causality	100%	–

Table 4: Fixed-budget residency pressure over five seeds. Reload amplification is physical loads divided by requests; lower is better.

Policy	Mean loads	Mean evictions	Reload amplification
lru	77.2	73.8	0.643
size_aware_lru	47.6	43.6	0.397
reuse_count	53.4	49.4	0.445
reload_cost	52.4	48.4	0.437

7 Promotion Gates

Gate	Status	Evidence
environment	closed	vLLM V1 with native Metal paged attention runs reproducibly
black-box semantics	smoke closed	selected and full context agree on the fixed task
prefix coexistence	smoke closed	APC remains active; repeated prompts are fast
native block attention	controlled closed	80 Metal kernel rows; dense-reference parity
physical sharing	controlled closed	one immutable handle at concurrency 1–8
V1 metadata contract	controlled closed	selected block tables and positions remain scheduler-disjoint
live V1 generation	controlled closed	positive, disabled, wrong-memory, and cleanup controls over five seeds
engine-native serving	partial	live generation measured; APC coexistence, concurrency, connectors remain open
robustness	open	one model family, one kernel backend, controlled tensors

The Metal backend now accepts selected non-prefix K/V at its native paged-attention boundary and carries scheduler-invisible pages through V1 generation. Remaining boundaries are APC coexistence in the same selected request, continuous-batching concurrency, LMCache/connectors, and CUDA replication. The shared residency manager implements deterministic prefetch, pinning, sharing, and four eviction policies, but its pressure sweep is controlled simulation rather than vLLM scheduler telemetry.

8 Falsification and Limitations

The central thesis fails if an optimized E0/G10 integration matches an engine-aware implementation at equal quality across memory pressure, concurrency, and reuse. The present data make that falsifier stronger: the black-box baseline already preserves quality, uses 76.9% fewer visible prompt tokens than full context, and coexists with APC. A native implementation must improve resident memory, transfer, TTFT, or throughput beyond this baseline.

The current study uses a single small 4-bit model, one Apple host, synthetic answers, and five seeds. The cold requests are order-sensitive, per-request vLLM cached-token counts are unavailable, and no native APC-coexistence, continuous-batching concurrency, or HBM-pressure sweep was run. The live bridge requires page alignment, one K/V cache group, all consumer layers, and unquantized selected K/V. Apple unified memory also differs from the CUDA/LMCache environment for which many connector mechanisms were designed.

9 Related Work

vLLM introduced PagedAttention and continuous batching to reduce KV fragmentation and improve serving throughput [1]. Prefix caching reuses exact token histories; KV connectors and offload systems extend physical residency. Retrieval-augmented generation and sparse attention reduce the logical context presented to a model, but selected text is not equivalent to native non-prefix K/V. PRA combines explicit logical identity, retrieval, and native detail activation. The contribution here is the boundary between that logical mapping and vLLM's physical mapping.

10 Conclusion

Paper 6 establishes a reproducible vLLM-Metal baseline and a control plane that cannot accidentally overclaim native execution. Selected text recovers the fixed answer with 76.9% fewer visible tokens than full context and preserves APC. That success raises, rather than lowers, the bar for deep integration. The selected-K/V path is now verified at kernel level and in live V1 generation, including wrong-memory causality and post-request isolation. PRA-vLLM is therefore an engine-native controlled prototype on Metal. The next decisive experiment compares matched-quality residency, transfer, native APC coexistence, continuous-batching throughput, and a CUDA implementation.

References

[1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*, 2023.